COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	04/08/2026	**DATE SUBMITTED:**	04/08/2026
NAME:	Kherwin Emmanuel D. Milan	**INSTRUCTOR:**	

Hands-on Activity 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

```cpp
//ILOA
#include <iostream>
#include <stack>

int main() {

    std::stack<int> stack1;

    //Testing empty
    std::cout << "Testing empty(): "
              << stack1.empty() << std::endl;

    //Testing push
    stack1.push(10);
    stack1.push(20);
    stack1.push(30);

    std::cout << "After push(): "
              << stack1.top() << std::endl;

    //Testing top
    std::cout << "Testing top(): "
              << stack1.top() << std::endl;

    //Testing size
    std::cout << "Testing size(): "
              << stack1.size() << std::endl;
```

```
    //Testing pop
    stack1.pop();

    std::cout << "After pop(), top is: "
              << stack1.top() << std::endl;

    //Testing size again
    std::cout << "Size after pop(): "
              << stack1.size() << std::endl;

    //Testing emplace
    stack1.emplace(40);

    std::cout << "After emplace(), top is: "
              << stack1.top() << std::endl;

    //Creating another stack for swap
    std::stack<int> stack2;

    stack2.push(100);
    stack2.push(200);

    //Testing swap
    stack1.swap(stack2);

    std::cout << "After swap(), stack1 top is: "
              << stack1.top() << std::endl;

    return 0;
}
```

```
Testing empty(): 1
After push(): 30
Testing top(): 30
Testing size(): 3
After pop(), top is: 20
Size after pop(): 2
After emplace(), top is: 40
After swap(), stack1 top is: 200
PS C:\Users\kemil\OneDrive\Desktop\DSA>
```

```cpp
//ILOB1
#include <iostream>

//GLOBAL DECLARATION
#define maxCap 10

const size_t arrSize = 10;
int stackArr[maxCap];
int top = 1, newData;

//PROTOTYPE FUNCTIONS
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void display();

int main() {

    int choice;
    //MAIN DRIVER VROOM VROOM
    while(true){
        std::cout <<
"=============================================================================" <<
std::endl;
        std::cout << "Stack Operations \n" <<  std::endl;
        std::cout <<
"1.PUSH\n2.POP\n3.TOP\n4.isEmpty\n5.isFull\n6.Display" << std::endl;
        std::cout <<
"=============================================================================" <<
std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push(); break;
            case 2: pop(); break;
            case 3: Top(); break;
```

```
            case 4: std::cout << "Is stack empty? " << isEmpty() <<
std::endl; break;
            case 5: std::cout << "Is stack full? " << isFull() <<
std::endl; break;
            case 6: display(); break;
            default: std::cout << "Unalived Choice." << std::endl;
            break;
        }
    }
}

//FUNCTION DEFINITION

bool isEmpty() {
    /* HOW TO VERIFY IF STACK IS EMTPY */
    if(top == -1) return true;
    return false;
}

bool isFull(){
    if(top == maxCap - 1) return true;
    return false;
}

void push(){
    //ERROR CHECKING
    if(isFull()){
        std::cout << "Stack Overflow " << std::endl;
        return;
    }
    //PUSHING STACK
    std::cout << "\nEnter a new Value: " << std::endl;
    std::cin >> newData;
    //HOW TO INSERT THE DATA IN THE STACK
    stackArr[++top] = newData;
}

void pop(){
```

```
    //ERROR CHECKING
    if(isEmpty){
        std::cout << "Stack Underflow" << std::endl;
    }

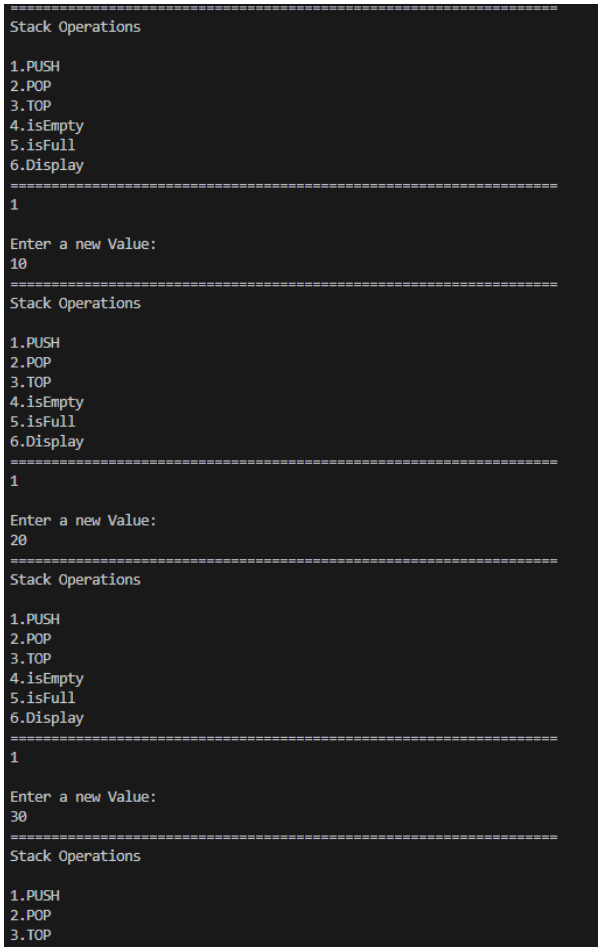
    //DISPLAY THE VALUE THAT WE ARE GOING TO POP IN POP IN BLOOM
    std::cout << "POPPING: " << stackArr[top] << std::endl;

    //DECREMENT THE TOP VALUE FRPM THE STACK
    top--;
}

void Top(){
    //ERROR CATCHING
    if(isEmpty){
        std::cout << "The Stack is empty" << std::endl;
        return;
    }
    //CHECK TOP VALUE
    std::cout << "Top Element: " << stackArr[top] << std::endl;
}

void display() {
    if (isEmpty()) {
        std::cout << "Stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (TOP TO BOT):" << std::endl;
    for (int j = top; j >= 0; j--) {
        std::cout << stackArr[j] << " ";
    }
    std::cout << std::endl;
}
```






```cpp
//ILOB2
#include <iostream>

class Node {
public:
    int data;
    Node* next;
};
Node* head = NULL;
Node* tail = NULL;


void push(int newData) {
    Node* newNode = new Node;
    newNode->data = newData;
```

```
    newNode->next = head;
    if (head == NULL) {
        head = tail = newNode;
    }
    else {
        newNode->next = head;
        head = newNode;
    }
}


int pop() {
    int tempVal;
    Node* temp;
    if (head == NULL) {
        head = tail = NULL;
        std::cout << "Stack Underflow."
                  << std::endl;
        return -1;
    }
    temp = head;
    tempVal = temp->data;
    head = head->next;
    delete temp;
    return tempVal;
}


void Top() {
    if (head == NULL) {
        std::cout << "Stack is Empty."
                  << std::endl;
        return;
    }
    std::cout << "Top of Stack: "
              << head->data
              << std::endl;
}
```

```
void display() {
    if (head == NULL) {
        std::cout << "Stack is Empty."
                  << std::endl;
        return;
    }
    Node* current = head;
    std::cout << "Stack elements: ";
    while (current != NULL) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}


int main() {
    push(1);
    std::cout << "After the first PUSH: ";
    Top();
    push(5);
    std::cout << "After the second PUSH: ";
    Top();
    std::cout << "\nAll elements: ";
    display();
    pop();
    std::cout << "After the first POP: ";
    Top();
    pop();
    std::cout << "After the second POP: ";
    Top();
    pop();

    return 0;
}
```

```
After the first PUSH: Top of Stack: 1
After the second PUSH: Top of Stack: 5

All elements: Stack elements: 5 1
After the first POP: Top of Stack: 1
After the second POP: Stack is Empty.
Stack Underflow.
PS C:\Users\kemil\OneDrive\Desktop\DSA>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```cpp
#include <iostream>
#include <string>

const int MAX = 100;

char stack[MAX];
int top = -1;

//Check if stack is empty
bool isEmpty() {
    return top == -1;
}

//Push a symbol into the stack
void push(char value) {
    if (top == MAX - 1) {
        std::cout << "Stack Overflow." << std::endl;
        return;
    }

    top++;
    stack[top] = value;
}

//Pop a symbol from the stack
char pop() {
    if (isEmpty()) {
```

```
        return '\0';
    }

    char value = stack[top];
    top--;

    return value;
}

//Check if opening and closing symbols match
bool isMatching(char opening, char closing) {

    if (opening == '(' && closing == ')')
        return true;

    if (opening == '[' && closing == ']')
        return true;

    if (opening == '{' && closing == '}')
        return true;

    return false;
}

//Check if symbols are balanced
bool checkBalanced(std::string expression) {

    //Reset the stack
    top = -1;

    //Read each character
    for (char character : expression) {

        //If opening symbol, push onto stack
        if (character == '(' ||
            character == '[' ||
            character == '{') {
```

```
            push(character);
        }

        //If closing symbol
        else if (character == ')' ||
                 character == ']' ||
                 character == '}') {

            //Report error if stack is empty
            if (isEmpty()) {
                return false;
            }

            //Pop the opening symbol
            char opening = pop();

            //Check if symbols match
            if (!isMatching(opening, character)) {
                return false;
            }
        }

        //Other characters are ignored
    }

    //If stack is not empty, there is an unmatched symbol
    return isEmpty();
}


int main() {

    std::string expression;

    std::cout << "Enter expression: ";
    std::getline(std::cin, expression);

    if (checkBalanced(expression)) {
```

```
        std::cout << "The symbols are balanced." << std::endl;
    }
    else {
        std::cout << "The symbols are not balanced." << std::endl;
    }

    return 0;
}
```

```
Enter expression: (A+B)+(C-D)
The symbols are balanced.
```

```
Enter expression: ((A+B)+(C-D)
The symbols are not balanced.
```

```
Enter expression: ((A+B)+[C-D])
The symbols are balanced.
```

```
Enter expression: ((A+B]+[C-D]}
The symbols are not balanced.
```

```
//ILOCp2
#include <iostream>
#include <string>

class Node {
public:
    char data;
    Node* next;
};

Node* top = nullptr;


//Check if stack is empty
bool isEmpty() {
    return top == nullptr;
}


//Push a symbol into the stack
void push(char value) {
```

```
    Node* newNode = new Node;

    newNode->data = value;
    newNode->next = top;

    top = newNode;
}


//Pop a symbol from the stack
char pop() {
    if (isEmpty()) {
        return '\0';
    }
    Node* temporary = top;
    char value = temporary->data;
    top = top->next;
    delete temporary;
    return value;
}


//Check if opening and closing symbols match
bool isMatching(char opening, char closing) {
    if (opening == '(' && closing == ')')
        return true;
    if (opening == '[' && closing == ']')
        return true;
    if (opening == '{' && closing == '}')
        return true;
    return false;
}


//Check if symbols are balanced
bool checkBalanced(std::string expression) {
    //Make sure the stack starts empty
```

```
    top = nullptr;

    //Read each character
    for (char character : expression) {
        //If opening symbol, push onto stack
        if (character == '(' ||
            character == '[' ||
            character == '{') {
            push(character);
        }
        //If closing symbol
        else if (character == ')' ||
                 character == ']' ||
                 character == '}') {
            //Report error if stack is empty
            if (isEmpty()) {
                return false;
            }
            //Pop the stack
            char opening = pop();
            //Check if symbols match
            if (!isMatching(opening, character)) {
                return false;
            }
        }
    }

    //At the end stack should be empty
    return isEmpty();
}

int main() {
    std::string expression;
    std::cout << "Enter expression: ";
    std::getline(std::cin, expression);
    if (checkBalanced(expression)) {
        std::cout << "The symbols are balanced." << std::endl;
    }
```

```
    else {
        std::cout << "The symbols are not balanced." << std::endl;
    }
    return 0;
}
```

```
Enter expression: (A+B)+(C-D)
The symbols are balanced.
```

```
Enter expression: ((A+B)+(C-D)
The symbols are not balanced.
```

```
Enter expression: ((A+B)+[C-D])
The symbols are balanced.
```

```
Enter expression: ((A+B]+[C-D]}
The symbols are not balanced.
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter expression: ((A+B)+[C-D]) The symbols are balanced.	All opening and closing parentheses have matching pairs.
((A+B)+(C-D)	N	Enter expression: ((A+B)+(C-D) The symbols are not balanced.	There is one opening parenthesis without a matching closing parenthesis.
((A+B)+[C-D])	Y	Enter expression: ((A+B)+[C-D]) The symbols are balanced.	All parentheses and brackets are properly matched.
((A+B]+[C-D]}	N	Enter expression: ((A+B]+[C-D]} The symbols are not balanced.	The opening and closing symbols do not match correctly.

Conclusion:
In this activity, I learned that a stack is a linear data structure that follows the LIFO (Last In, First Out) principle. I learned how to use the basic operations of a stack such as push, pop, top, and isEmpty. I also learned that stacks can be implemented using arrays, linked lists.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.